МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Компьютерные науки и прикладная математика»

Лабораторная работа № 4
по курсу «Численные методы»

Группа: М8О-303Б-23

Студент(ка): Жуховицкий А. Д.

Преподаватель: Ревизников Д.Л.

Оценка:

Москва, 2026

ОГЛАВЛЕНИЕ

1 Тема.....3
2 Задание 3
3 Ход лабораторной работы.....4

1 Тема

Численные методы решения нелинейных дифференциальных уравнений.

2 Задание

4.1. Реализовать методы Эйлера, Рунге-Кутты и Адамса 4-го порядка в виде программ, задавая в качестве входных данных шаг сетки h. С использованием разработанного программного обеспечения решить задачу Коши для ОДУ 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

7	$y''-4xy'+(4x^2-2)y=0$, $y(0)=1$, $y'(0)=1$, $x\in[0,1], h=0.1$	$y=(1+x)e^{x^2}$

4.2. Реализовать метод стрельбы и конечно-разностный метод решения краевой задачи для ОДУ в виде программ. С использованием разработанного программного обеспечения решить краевую задачу для обыкновенного дифференциального уравнения 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

7	(2x+1) y″+4xy′-4y=0, y′ (0)= –1, y′(1)+2y(1)=3	$y(x)=x+e^{-2x}$

3 Ход лабораторной работы

```
import numpy as np
import seaborn as sns


y = lambda x: (1 + x) * np.exp(x * x)

start, end = 0, 1
eps = 1e-3
h = 0.1

x = np.arange(start, end + h, h)
```

$$\begin{cases} y'' - 4xy' + (4x^2 - 2)y = 0, \\ y(0) = 1, \\ y'(0) = 1, \\ x \in [0, 1], \\ h = 0.1, \end{cases} \qquad y = (1 + x)e^{x^2}$$

Метод Эйлера второго порядка точности

Преобразуем дифференциальное уравнение второго порядка к системе двух уравнений первого порядка, вводя $z = y'$:

$$y' = z, \qquad z' = 4xz - (4x^2 - 2)y.$$

Также посчитаем значения решения на заданной сетке:

```
f_der_1 = lambda z: z
f_der_2 = lambda x, y, z: 4 * x * z - (4 * x * x - 2) * y

y_etalon_f = y(x)
```

Определим функцию, которая считает приближённое решение системы методом Эйлера второго порядка точности для начальных условий $y(x_0) = y_0$ и $z(x_0) = z_0$ с фиксированным шагом h:

```
def euler(y_der_1, y_der_2, x, y0, z0, h):
    y = np.zeros(len(x), dtype=np.float64)
    z = np.zeros(len(x), dtype=np.float64)
    y[0], z[0] = y0, z0

    for i in range(1, len(x)):
        x_k_1_2 = x[i - 1] + h / 2
        y_k_1_2 = y[i - 1] + h / 2 * y_der_1(z[i - 1])
        z_k_1_2 = z[i - 1] + h / 2 * y_der_2(x[i - 1], y[i - 1], z[i - 1])

        y[i] = y[i - 1] + h * y_der_1(z_k_1_2)
        z[i] = z[i - 1] + h * y_der_2(x_k_1_2, y_k_1_2, z_k_1_2)
```

```
    return y, z
```

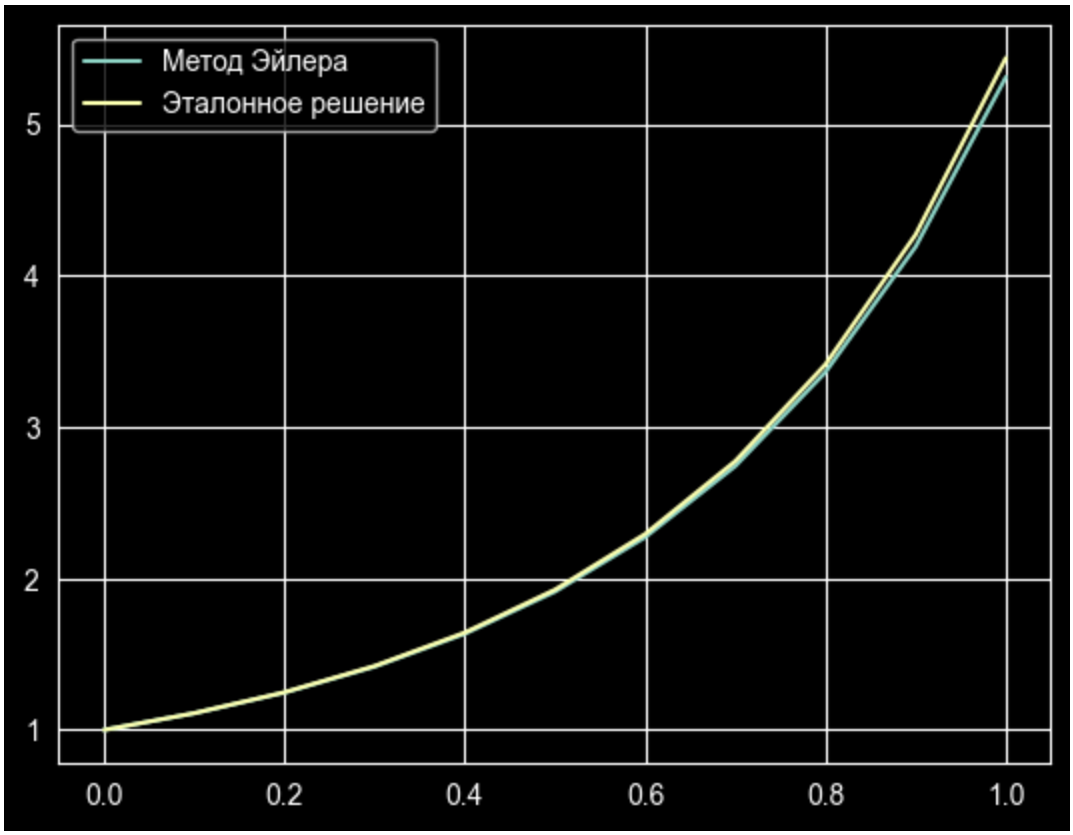
Посчитаем приближённые значения y и z методом Эйлера второго порядка точности на отрезке $[start, end]$ с шагом h:

```
y_euler, z_euler = euler(f_der_1, f_der_2, x, 1, 1, h)
```

Построим графики приближённого решения, полученного методом Эйлера, и эталонного решения и проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_euler, label='Метод Эйлера')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Рунге-Кутта четвёртого порядка точности

Определим функцию, которая считает приближённое решение системы методом Рунге-Кутта четвёртого порядка точности для начальных условий $y(x_0) = y_0$ и $z(x_0) = z_0$ с фиксированным шагом h:

```
def runge_kutta(y_der_1, y_der_2, x, y0, z0, h):
    y = np.zeros(len(x), dtype=np.float64)
    z = np.zeros(len(x), dtype=np.float64)

    y[0], z[0] = y0, z0
```

```
    for i in range(1, len(x)):
        K_k_1 = h * y_der_1(z[i - 1])
        L_k_1 = h * y_der_2(x[i - 1], y[i - 1], z[i - 1])


        K_k_2 = h * y_der_1(z[i - 1] + L_k_1 / 2)

        L_k_2 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_1 / 2,
            z[i - 1] + L_k_1 / 2
        )


        K_k_3 = h * y_der_1(z[i - 1] + L_k_2 / 2)

        L_k_3 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_2 / 2,
            z[i - 1] + L_k_2 / 2
        )


        K_k_4 = h * y_der_1(z[i - 1] + L_k_3)

        L_k_4 = h * y_der_2(
            x[i - 1] + h,
            y[i - 1] + K_k_3,
            z[i - 1] + L_k_3
        )


        y_delta = (K_k_1 + 2 * K_k_2 + 2 * K_k_3 + K_k_4) / 6
        z_delta = (L_k_1 + 2 * L_k_2 + 2 * L_k_3 + L_k_4) / 6


        y[i] = y[i - 1] + y_delta
        z[i] = z[i - 1] + z_delta

    return y, z
```

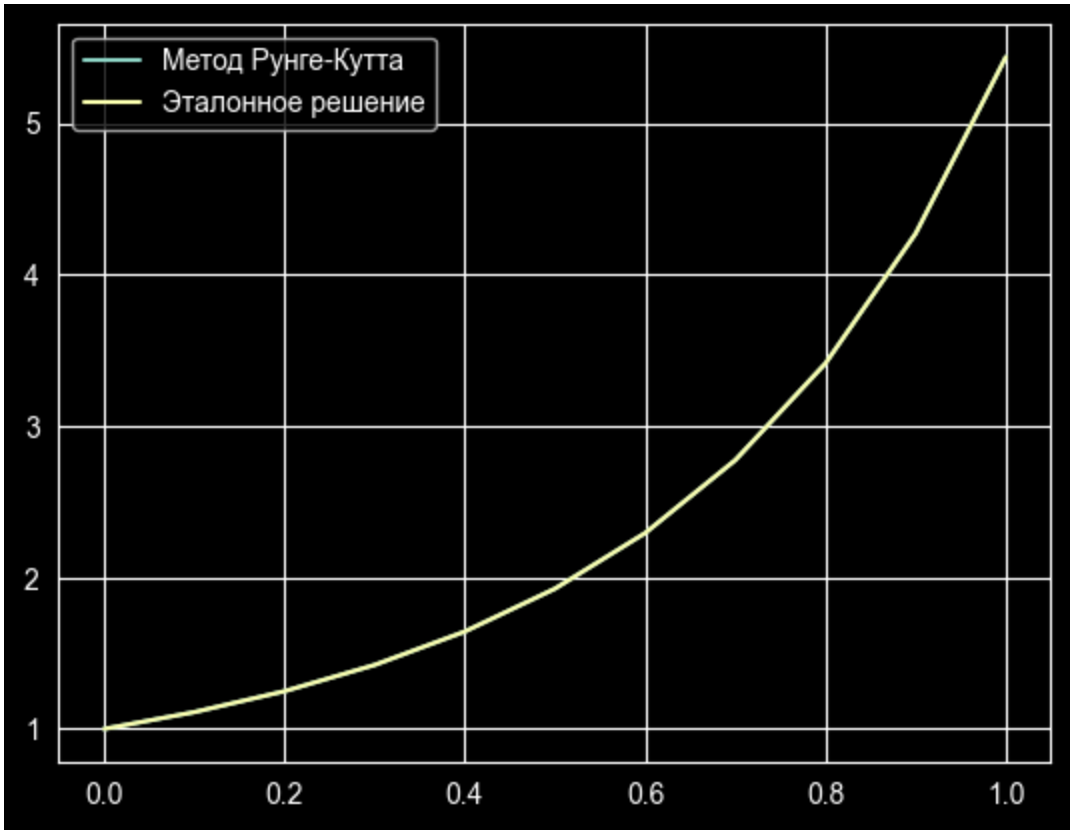
Посчитаем приближённые значения y и z методом Рунге-Кутта четвёртого порядка точности на отрезке $[start, end]$ с шагом h:

```
y_runge_kutta, z_runge_kutta = runge_kutta(f_der_1, f_der_2, x, 1, 1, h)
```

Построим графики приближённого решения, полученного методом Рунге-Кутта, и эталонного решения. Проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_runge_kutta, label='Метод Рунге-Кутта')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Адамса четвёртого порядка точности

Определим функцию, которая считает приближённое решение системы явным методом Адамса четвёртого порядка точности. Первые четыре значения берутся из уже найденного решения методом Рунге-Кутта:

```
def adams(y_der_1, y_der_2, x, y_start, z_start, h):
    y = np.concat((y_start, np.zeros(len(x) - 4, dtype=np.float64)))
    z = np.concat((z_start, np.zeros(len(x) - 4, dtype=np.float64)))


    for i in range(4, len(x)):
        y[i] = y[i - 1] + (h / 24) * (
            55 * y_der_1(z[i - 1])
            - 59 * y_der_1(z[i - 2])
            + 37 * y_der_1(z[i - 3])
            - 9 * y_der_1(z[i - 4])
        )

        z[i] = z[i - 1] + (h / 24) * (
            55 * y_der_2(x[i - 1], y[i - 1], z[i - 1])
            - 59 * y_der_2(x[i - 2], y[i - 2], z[i - 2])
            + 37 * y_der_2(x[i - 3], y[i - 3], z[i - 3])
            - 9 * y_der_2(x[i - 4], y[i - 4], z[i - 4])
        )

    return y, z
```

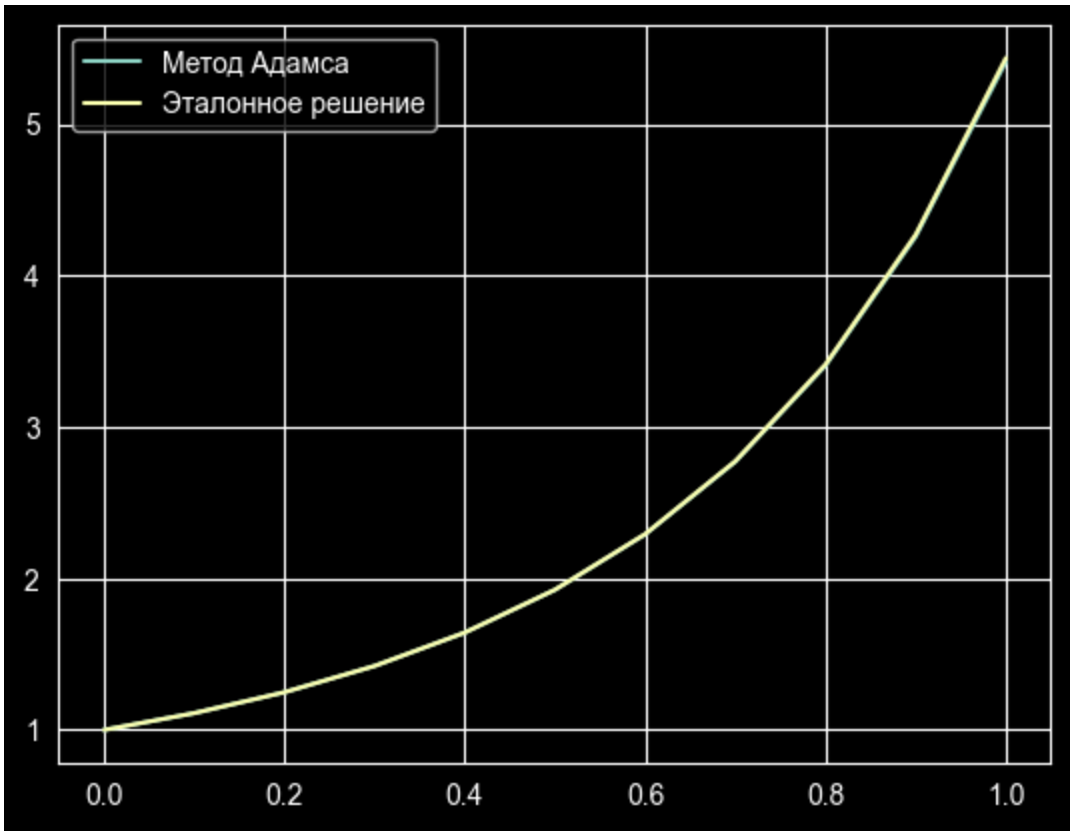
Посчитаем приближённые значения y и z методом Адамса четвёртого порядка точности, используя первые четыре точки решения, полученного методом Рунге-Кутта:

```
y_adams, z_adams = adams(
    f_der_1,
    f_der_2,
    x,
    y_runge_kutta[:4],
    z_runge_kutta[:4],
    h
)
```

Построим графики приближённого решения, полученного методом Адамса, и эталонного решения. Проверим корректность реализованного метода:

```
sns.lineplot(x=x, y=y_adams, label='Метод Адамса')
sns.lineplot(x=x, y=y_etalon_f, label='Эталонное решение')
```

```
<Axes: >
```



Метод Рунге-Ромберга-Ричардсона для контроля точности

Определим функцию, которая считает приближённое значение погрешности по формуле метода Рунге-Ромберга-Ричардсона для решений, полученных с шагами h и kh, при порядке точности p:

```
def runge_romberg_richardson(f_h, f_kh, p, k_coef):
    return (f_h[::k_coef] - f_kh) / (k_coef ** p - 1)
```

Уменьшим шаг в k раз, построим новую сетку и повторно найдём приближённые решения методами Эйлера, Рунге-Кутта и Адамса:

```
k = 2
smaller_h = h / k
x_smaller_h = np.arange(start, end + smaller_h, smaller_h)

y_euler_smaller_h, z_euler_smaller_h = euler(
    f_der_1, f_der_2, x_smaller_h,
    1, 1, smaller_h
)
y_runge_kutta_smaller_h, z_runge_kutta_smaller_h = runge_kutta(
    f_der_1, f_der_2, x_smaller_h,
    1, 1, smaller_h
)
y_adams_smaller_h, z_adams_smaller_h = adams(
    f_der_1, f_der_2, x_smaller_h,
    y_runge_kutta_smaller_h[:4], z_runge_kutta_smaller_h[:4], smaller_h
)
```

Найдём приближённые значения погрешности по формуле метода Рунге-Ромберга-Ричардсона для методов Эйлера, Рунге-Кутта и Адамса:

```
euler_runge_romberg = runge_romberg_richardson(
    y_euler_smaller_h,
    y_euler, 2, k
)
runge_kutta_runge_romberg = runge_romberg_richardson(
    y_runge_kutta_smaller_h,
    y_runge_kutta, 4, k
)
adams_runge_romberg = runge_romberg_richardson(
    y_adams_smaller_h,
    y_adams, 4, k
)
```

Определим функцию, которая считает суммарную абсолютную погрешность приближённого решения относительно эталонного решения:

```
def error(v, v_etalon):
    return np.sum(np.abs(v - v_etalon))
```

Уточним приближённые решения, добавив к ним оценки погрешностей, полученные по формуле Рунге-Ромберга-Ричардсона:

```
y_euler_runge_romberg = y_euler + euler_runge_romberg
y_runge_kutta_runge_romberg = y_runge_kutta + runge_kutta_runge_romberg
y_adams_runge_romberg = y_adams + adams_runge_romberg
```

Валидация

Проверим максимальные по модулю оценки погрешности для методов Эйлера, Рунге-Кутта и Адамса, чтобы убедиться в корректности их подсчёта:

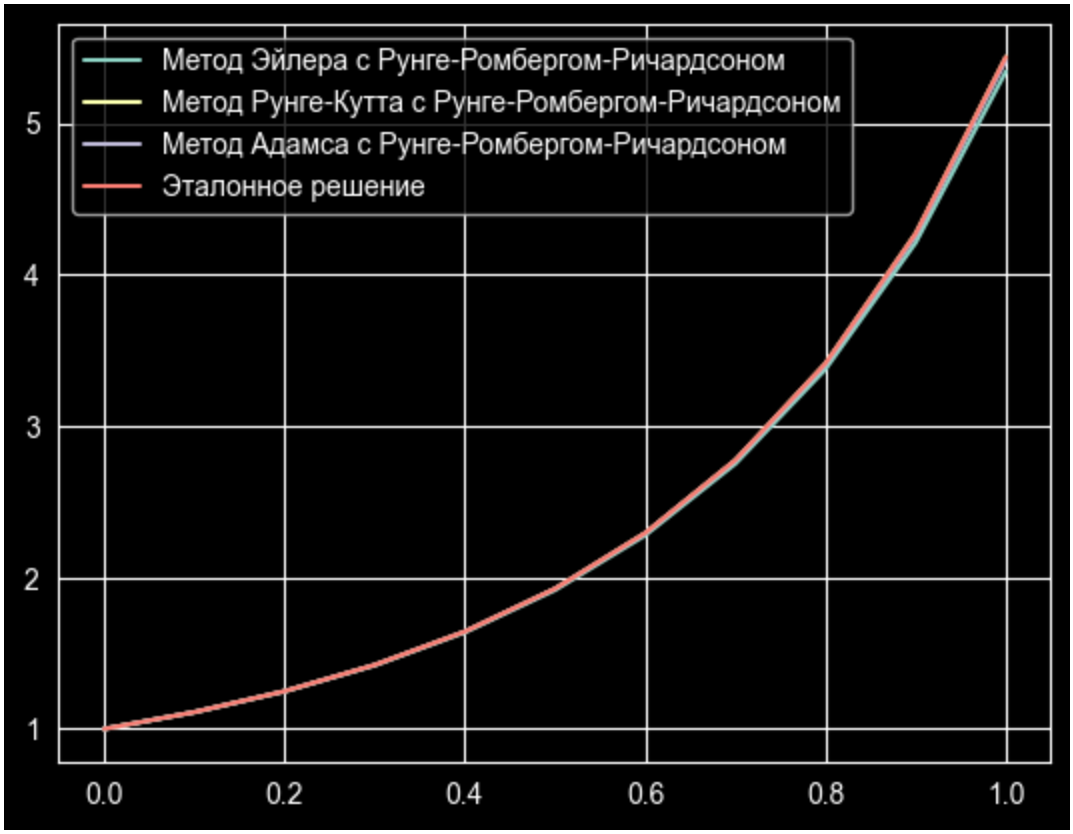
```
print(
    np.max(np.abs(euler_runge_romberg)),
    np.max(np.abs(runge_kutta_runge_romberg)),
    np.max(np.abs(adams_runge_romberg)),
)
```

```
0.030417192786919678 2.568946429786223e-05 0.001526943109977689
```

Построим графики уточнённых решений после применения метода Рунге-Ромберга-Ричардсона и сравним их с эталонным решением:

```
sns.lineplot(
    x=x, y=y_euler_runge_romberg,
    label='Метод Эйлера с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_runge_kutta_runge_romberg,
    label='Метод Рунге-Кутта с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_adams_runge_romberg,
    label='Метод Адамса с Рунге-Ромбергом-Ричардсоном'
)
sns.lineplot(
    x=x, y=y_etalon_f,
    label='Эталонное решение'
)
```

```
<Axes: >
```



Посчитаем суммарные абсолютные погрешности уточнённых решений относительно эталонного решения:

```
print(
    "Погрешность метода Эйлера с Рунге-Ромбергом-Ричардсоном:",
    error(y_euler_runge_romberg, y_etalon_f)
)
print(
    "Погрешность метода Рунге-Кутта с Рунге-Ромбергом-Ричардсоном:",
    error(y_runge_kutta_runge_romberg, y_etalon_f)
)
print(
    "Погрешность метода Адамса с Рунге-Ромбергом-Ричардсоном:",
    error(y_adams_runge_romberg, y_etalon_f)
)
```

```
Погрешность метода Эйлера с Рунге-Ромбергом-Ричардсоном: 0.25910783314719277
Погрешность метода Рунге-Кутта с Рунге-Ромбергом-Ричардсоном: 0.0009050781446588996
Погрешность метода Адамса с Рунге-Ромбергом-Ричардсоном: 0.053685829495393245
```

```
import numpy as np
import seaborn as sns


eps = 1e-3
h = 0.1

start, end = 0, 1
x = np.arange(start, end + h, h)
y = lambda x: x + np.exp(-2 * x)
```

$$\begin{cases} (2x+1)\,y'' + 4x\,y' - 4y = 0, \\ y'(0) = -1, \\ y'(1) + 2y(1) = 3. \end{cases} \qquad y(x) = x + e^{-2x}$$

Преобразуем дифференциальное уравнение второго порядка к системе двух уравнений первого порядка, вводя $z = y'$:

$$y' = z, \qquad z' = \frac{4y - 4xz}{2x+1}.$$

```
def error(v, v_etalon):
    return np.sum(np.abs(v - v_etalon))
```

```
def runge_romberg_richardson(f_h, f_kh, p, k_coef):
    return (f_h[::k_coef] - f_kh) / (k_coef ** p - 1)
```

Метод стрельбы

Зададим функции правых частей системы:

$$y' = z, \qquad z' = \frac{4y - 4xz}{2x+1}.$$

Также вычислим значения точного решения на основной сетке.

```
f_der_1 = lambda z: z
f_der_2 = lambda x, y, z: (4 * y - 4 * x * z) / (2 * x + 1)

y_etalon_f = y(x)
```

Метод Рунге–Кутты 4-го порядка для задачи Коши

```
def runge_kutta(y_der_1, y_der_2, x, y0, z0, h):
    y = np.zeros(len(x), dtype=np.float64)
    z = np.zeros(len(x), dtype=np.float64)

    y[0], z[0] = y0, z0

    for i in range(1, len(x)):
        K_k_1 = h * y_der_1(z[i - 1])
```

```
        L_k_1 = h * y_der_2(x[i - 1], y[i - 1], z[i - 1])


        K_k_2 = h * y_der_1(z[i - 1] + L_k_1 / 2)

        L_k_2 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_1 / 2,
            z[i - 1] + L_k_1 / 2
        )


        K_k_3 = h * y_der_1(z[i - 1] + L_k_2 / 2)

        L_k_3 = h * y_der_2(
            x[i - 1] + h / 2,
            y[i - 1] + K_k_2 / 2,
            z[i - 1] + L_k_2 / 2
        )


        K_k_4 = h * y_der_1(z[i - 1] + L_k_3)

        L_k_4 = h * y_der_2(
            x[i - 1] + h,
            y[i - 1] + K_k_3,
            z[i - 1] + L_k_3
        )


        y_delta = (K_k_1 + 2 * K_k_2 + 2 * K_k_3 + K_k_4) / 6
        z_delta = (L_k_1 + 2 * L_k_2 + 2 * L_k_3 + L_k_4) / 6


        y[i] = y[i - 1] + y_delta
        z[i] = z[i - 1] + z_delta

    return y, z
```

Неизвестное начальное значение обозначается как:

$$y(0) = s.$$

Для каждого s решается задача Коши:

$$y(0) = s, \qquad z(0) = y'(0) = -1.$$

После интегрирования до (x=1) проверяется правое граничное условие:

$$z(1) + 2y(1) = 3.$$

Невязка имеет вид:

$$\Phi(s) = z(1) + 2y(1) - 3.$$

Реализуем метод стрельбы:

```
def shooting_method(y_der_1, y_der_2, x, h, s0, s1, z0, eps=1e-3):

    def phi(s):
        y_tmp, z_tmp = runge_kutta(y_der_1, y_der_2, x, s, z0, h)
        return z_tmp[-1] + 2 * y_tmp[-1] - 3

    phi0 = phi(s0)
    phi1 = phi(s1)


    while True:
        if abs(phi1) <= eps:
            break

        if abs(phi1 - phi0) < 1e-15:
            raise ZeroDivisionError(
                "Метод секущих остановлен: разность невязок слишком мала."
            )

        s2 = s1 - phi1 * (s1 - s0) / (phi1 - phi0)
        phi2 = phi(s2)

        s0, phi0 = s1, phi1
        s1, phi1 = s2, phi2

    y_result, z_result = runge_kutta(y_der_1, y_der_2, x, s1, z0, h)

    return s1, y_result, z_result
```

В качестве двух начальных приближений для неизвестного $(s = y(0))$ положим `s0 = 0` и `s1 = 2`, как пробные значения функции в левой точке. Из условия известно $(z(0) = y'(0) = -1)$.

Решим и провалидируем основные моменты:

```
s0 = 0.0
s1 = 2.0

z0 = -1.0

s, y_shooting, z_shooting = shooting_method(
    f_der_1,
    f_der_2,
    x,
    h,
    s0,
    s1,
    z0,
    eps
)

print(f"Найденное начальное значение y(0) = {s:.3f}")
print(np.isclose(y_etalon_f[0], s, eps))
```

```
print(f"Невязка правого условия = {z_shooting[-1] + 2 * y_shooting[-1] - 3:.3e}")
print(np.isclose(z_shooting[-1] + 2 * y_shooting[-1], 3, eps))
```

```
Найденное начальное значение y(0) = 1.000
True
Невязка правого условия = 4.441e-16
True
```

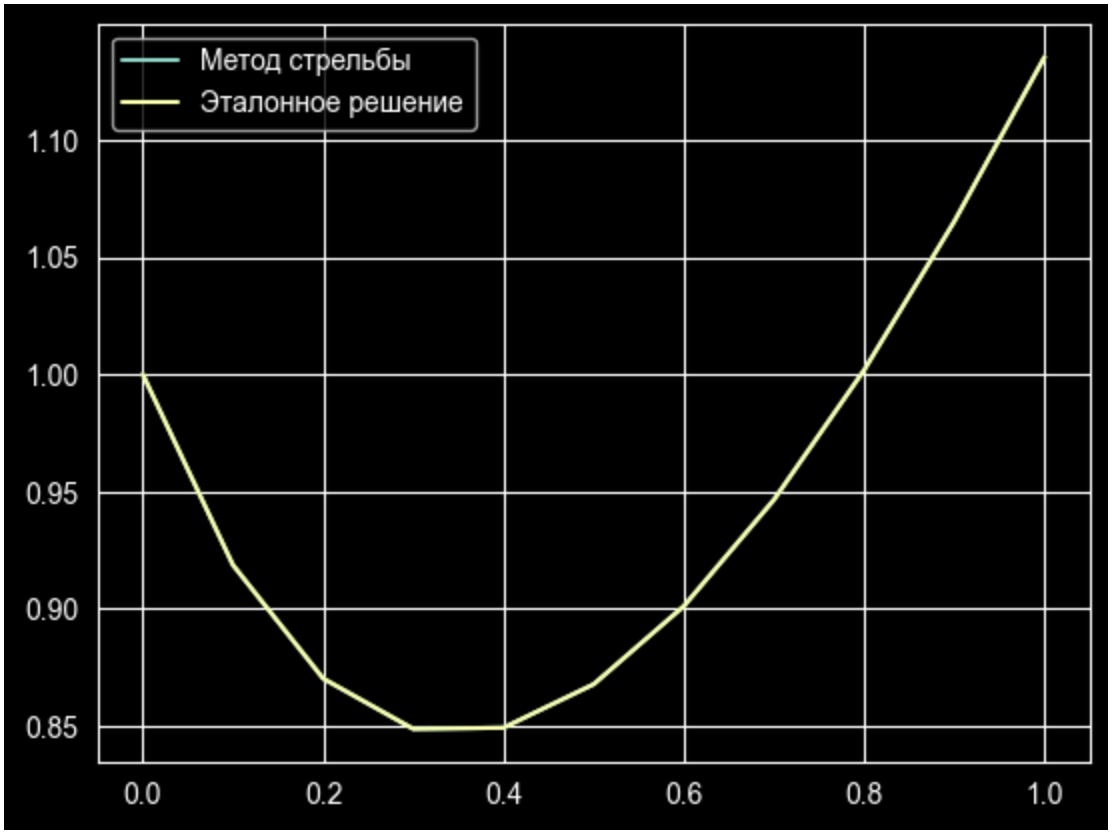
Погрешность метода стрельбы

```
print(error(y_shooting, y_etalon_f))
```

```
4.863503001784064e-05
```

```
sns.lineplot(x=x, y=y_shooting, label="Метод стрельбы")
sns.lineplot(x=x, y=y_etalon_f, label="Эталонное решение")
```

```
<Axes: >
```



Конечно-разностный метод

Коэффициенты стандартного вида уравнения

Сначала приводим уравнение к виду:

$$y'' + p(x)y' + q(x)y = 0.$$

Делим исходное уравнение на (2x+1):

$$y'' + \frac{4x}{2x+1}y' - \frac{4}{2x+1}y = 0.$$

Отсюда:

$p(x) = \frac{4x}{2x+1}, \qquad q(x) = -\frac{4}{2x+1}.$

```
p = (4 * x) / (2 * x + 1)
q = -4 / (2 * x + 1)
```

Левое граничное условие: $y'(0) = -1$

Аппроксимация: $\frac{y_1 - y_0}{h} = -1$

Итого: $-\frac{1}{h}y_0 + \frac{1}{h}y_1 = -1$

Внутренние узлы:

$$\left(1 - \frac{p_i h}{2}\right) y_{i-1} + \left(-2 + h^2 q_i\right) y_i + \left(1 + \frac{p_i h}{2}\right) y_{i+1} = 0$$

Правое граничное условие: $y'(1) + 2y(1) = 3$

Аппроксимация: $\frac{y_n - y_{n-1}}{h} + 2y_n = 3$

Итого:

$$-\frac{1}{h}y_{n-1} + \left(\frac{1}{h} + 2\right) y_n = 3$$

Реализуем конечно-разностный метод для данной задачи Коши, используя указанные аппроксимации для граничных условий и внутренних узлов. Используем метод прогонки для решения получившейся трёхдиагональной системы.

```
def finite_difference(x_sample, p, q, h):
    # Составляем матрицу для метода прогонки
    n = len(x_sample) - 1
    A = np.zeros((n + 1, 3))
    b = np.zeros(n + 1)

    for i in range(len(x_sample)):
        if i == 0:
            A[i][1] = -1 / h
            A[i][2] = 1 / h
            b[i] = -1
        elif i == n:
            A[i][0] = -1 / h
            A[i][1] = 1 / h + 2
            b[i] = 3
        else:
            A[i][0] = 1 - p[i] * h / 2
            A[i][1] = -2 + h ** 2 * q[i]
            A[i][2] = 1 + p[i] * h / 2
```

```
    # Метод прогонки из прошлых ЛР
    n = len(x_sample)
    P, Q = np.zeros(n + 1, dtype=float), np.zeros(n + 1, dtype=float)

    P[0], Q[0] = 0, 0

    for i in range(0, n):
        P[i + 1] = -A[i, 2] / (A[i, 1] + A[i, 0] * P[i])
        Q[i + 1] = (b[i] - A[i, 0] * Q[i]) / (A[i, 1] + A[i, 0] * P[i])

    y_finite_difference_result = np.zeros(n, dtype=float)

    for i in range(n, 0, -1):
        if P[i] == 0:
            y_finite_difference_result[i - 1] = Q[i]
            continue
        y_finite_difference_result[i - 1] = (
            P[i] * y_finite_difference_result[i] + Q[i]
        )

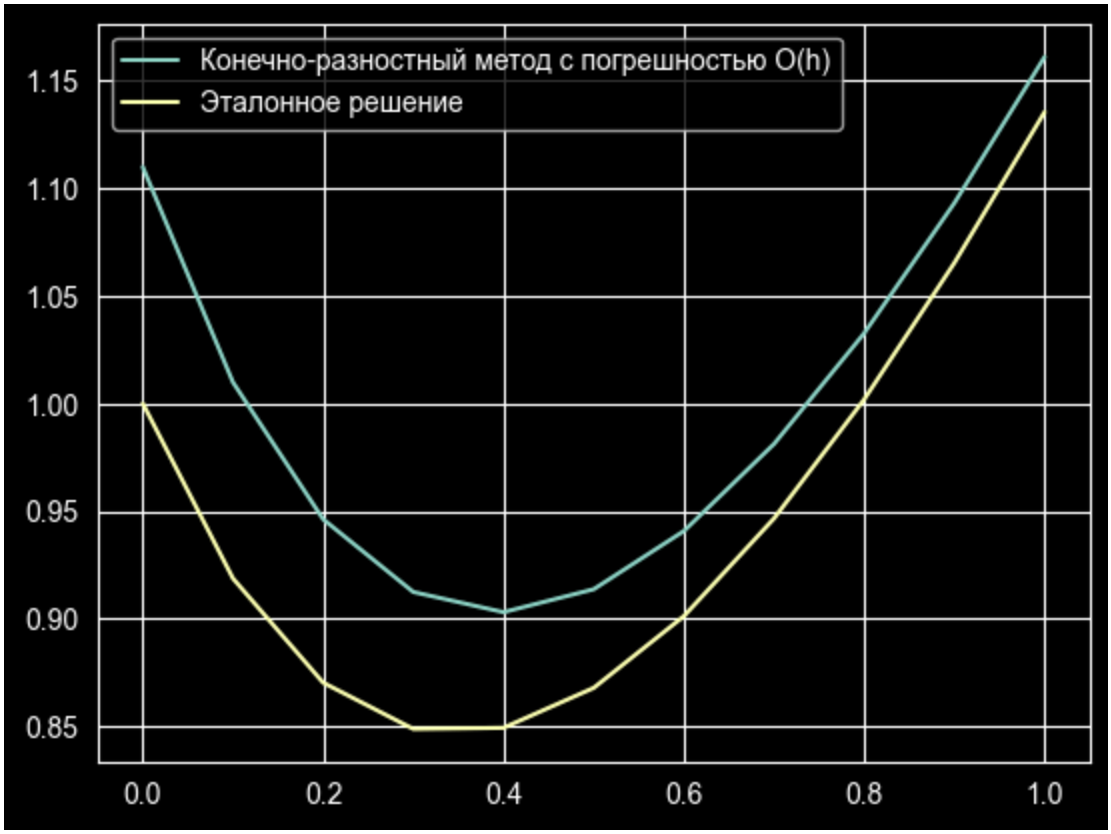
    return y_finite_difference_result
```

Рассчитаем приближённые значения y в узлах сетки, используя конечно-разностный метод:

```
y_finite_difference = finite_difference(x, p, q, h)
```

```
sns.lineplot(
    x=x, y=y_finite_difference,
    label="Конечно-разностный метод с погрешностью O(h)"
)
sns.lineplot(
    x=x, y=y_etalon_f,
    label="Эталонное решение"
)
```

```
<Axes: >
```



На графике видно, что конечно-разностный метод с погрешностью $O(h)$ даёт заметно более грубое приближение, чем метод стрельбы, который имеет погрешность $O(h^4)$.

Погрешность конечно-разностного метода

```
print(error(y_finite_difference, y_etalon_f))
```

```
0.5981383213006737
```

Валидация

Метод Рунге–Ромберга–Ричардсона

Для оценки главного члена погрешности одно и то же решение посчитаем на двух сетках: с шагом (h) и с шагом (h/2). После этого решения сравниваются в общих узлах.

```
k = 2
smaller_h = h / k
x_smaller_h = np.arange(start, end + smaller_h, smaller_h)
```

Прогоним решения тех же задач на сетке с шагом h/2

```
p_smaller_h = (4 * x_smaller_h) / (2 * x_smaller_h + 1)
q_smaller_h = -4 / (2 * x_smaller_h + 1)
```

```
_, y_shooting_smaller_h, z_shooting_smaller_h = shooting_method(
    f_der_1,
    f_der_2,
    x_smaller_h,
    smaller_h,
    s0,
    s1,
    z0,
    eps
)
y_finite_difference_smaller_h = finite_difference(
    x_smaller_h,
    p_smaller_h,
    q_smaller_h,
    smaller_h
)
```

Найдём погрешности по Рунге–Ромбергу

```
runge_romberg_shooting = runge_romberg_richardson(
    y_shooting_smaller_h,
    y_shooting, 4, k
)
runge_romberg_finite_difference = runge_romberg_richardson(
    y_finite_difference_smaller_h,
    y_finite_difference, 2, k
)
```

```
print(
    np.max(np.abs(runge_romberg_shooting)),
    np.max(np.abs(runge_romberg_finite_difference))
)
```

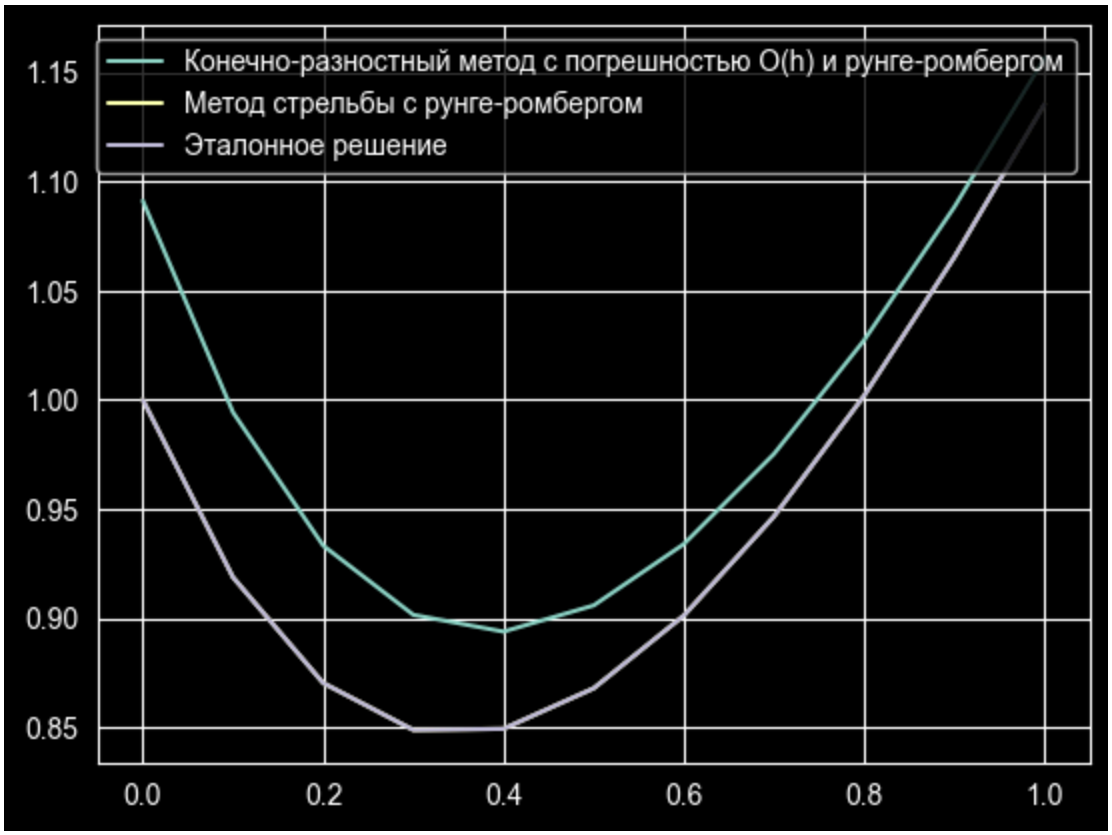
```
3.6424752023478393e-07 0.018745822451503653
```

Уточнение решений и итоговый график

```
shooting_with_runge_romberg = y_shooting + runge_romberg_shooting
finite_difference_with_runge_romberg = (
        y_finite_difference + runge_romberg_finite_difference
)
```

```
sns.lineplot(
    x=x, y=finite_difference_with_runge_romberg,
    label="Конечно-разностный метод с погрешностью O(h) и рунге-ромбергом"
)
sns.lineplot(
    x=x, y=shooting_with_runge_romberg,
    label="Метод стрельбы с рунге-ромбергом"
)
sns.lineplot(
    x=x, y=y_etalon_f,
    label="Эталонное решение"
)
```

<Axes: >


Конечно-разностный метод с погрешностью O(h) и рунге-ромбергом
Метод стрельбы с рунге-ромбергом
Эталонное решение
1.15
1.10
1.05
1.00
0.95
0.90
0.85
0.0
0.2
0.4
0.6
0.8
1.0